Romain Pereira

Rapport de Projet DevOps

Déploiement d'une infrastructure cloud avec Terraform

1. Introduction

Ce projet consiste à déployer une infrastructure cloud sur Azure en utilisant Terraform. L'idée c'est de tout automatiser : au lieu de créer les ressources à la main sur le portail Azure, on écrit des fichiers de configuration et Terraform s'occupe du reste. Concrètement, on déploie une VM avec un backend Flask dessus, un stockage pour les fichiers, et une base de données PostgreSQL.

Stack utilisée :

- Microsoft Azure comme provider cloud
- Terraform pour l'automatisation de l'infrastructure
- Flask (Python 3) pour le backend API REST
- Azure Blob Storage pour stocker les fichiers statiques
- PostgreSQL Flexible Server pour la base de données

2. Préparation de l'environnement

La première étape est d'installer les outils nécessaires. Sur Mac, Terraform et Azure CLI s'installent via Homebrew. Une fois installés, on se connecte à Azure depuis le terminal avec az login, ce qui ouvre le navigateur pour s'authentifier.

Les fichiers Terraform sont organisés de façon à séparer les responsabilités :

- provider.tf — définit le provider Azure et la version Terraform requise
- variables.tf — déclare toutes les variables avec leurs types et valeurs par défaut
- main.tf — contient toutes les ressources à créer
- outputs.tf — affiche les informations utiles après le déploiement (IP, URLs)
- terraform.tfvars — contient les valeurs sensibles, non pushé sur GitHub

La commande terraform init télécharge le provider Azure et initialise le projet.

```
→  terraform terraform init

Initializing the backend...

Initializing provider plugins...
- Finding hashicorp/azurerm versions matching "~> 3.0"...
- Installing hashicorp/azurerm v3.117.1...
- Installed hashicorp/azurerm v3.117.1 (signed by HashiCorp)

Terraform has created a lock file .terraform.lock.hcl to record the provider
selections it made above. Include this file in your version control repository
so that Terraform can guarantee to make the same selections by default when
you run "terraform init" in the future.

Terraform has been successfully initialized!

You may now begin working with Terraform. Try running "terraform plan" to see
any changes that are required for your infrastructure. All Terraform commands
should now work.

If you ever set or change modules or backend configuration for Terraform,
rerun this command to reinitialize your working directory. If you forget, other
commands will detect it and remind you to do so if necessary.
→  terraform
```

3. Déploiement

Ressources créées

La commande terraform apply lit les fichiers .tf et crée toutes les ressources dans l'ordre correct en gérant les dépendances automatiquement. Les ressources déployées sont :

- Resource Group devops-romain en France Central — conteneur logique pour toutes les ressources
- VM Ubuntu 22.04 Standard_D2s_v3 avec IP publique statique
- Virtual Network (10.0.0.0/16) et sous-réseau (10.0.1.0/24)
- Network Security Group avec règles d'accès SSH (port 22) et Flask (port 5000)
- Storage Account devopsromainstore avec un container Blob nommé fichiers
- Serveur PostgreSQL Flexible Server version 14

Problème 1 — Zone et SKU indisponibles

Au premier terraform apply, deux erreurs sont apparues. D'abord, la zone de disponibilité '1' n'était pas disponible dans la région West Europe pour cet abonnement Azure Efrei. Ensuite, le SKU Basic pour l'IP publique était aussi restreint sur cet abonnement.

```
Error: creating Flexible Server (Subscription: "c71f8ae4-544c-4fea-80b2-95d882c1405c"
Resource Group Name: "devops-romain"
Flexible Server Name: "devops-romain-db"): polling after Create: polling failed: the Azure API returned the follow
ing error:

Status: "AvailabilityZoneNotAvailable"
Code: ""
Message: "Availability zone '1' isn't available in location 'westeurope' for subscription 'c71f8ae4-544c-4fea-80b2
-95d882c1405c'. Choose a different availability zone."
Activity Id: ""

---

API Response:

----[start]----
{"name":"fe379e36-13f5-49bf-980d-7a4afd9d92df","status":"Failed","startTime":"2026-03-13T09:49:44.403Z","error":{"
code":"AvailabilityZoneNotAvailable","message":"Availability zone '1' isn't available in location 'westeurope' for s
ubscription 'c71f8ae4-544c-4fea-80b2-95d882c1405c'. Choose a different availability zone."}}
-----[end]-----


  with azurerm_postgresql_flexible_server.main,
  on main.tf line 23, in resource "azurerm_postgresql_flexible_server" "main":
  23: resource "azurerm_postgresql_flexible_server" "main" {


Error: updating Public I P Address (Subscription: "c71f8ae4-544c-4fea-80b2-95d882c1405c"
Resource Group Name: "devops-romain"
Public I P Addresses Name: "devops-romain-ip"): performing CreateOrUpdate: unexpected status 400 (400 Bad Request)
with error: IPv4BasicSkuPublicIpCountLimitReached: Cannot create more than 0 IPv4 Basic SKU public IP addresses for
this subscription in this region.

  with azurerm_public_ip.main,
  on main.tf line 57, in resource "azurerm_public_ip" "main":
  57: resource "azurerm_public_ip" "main" {
```

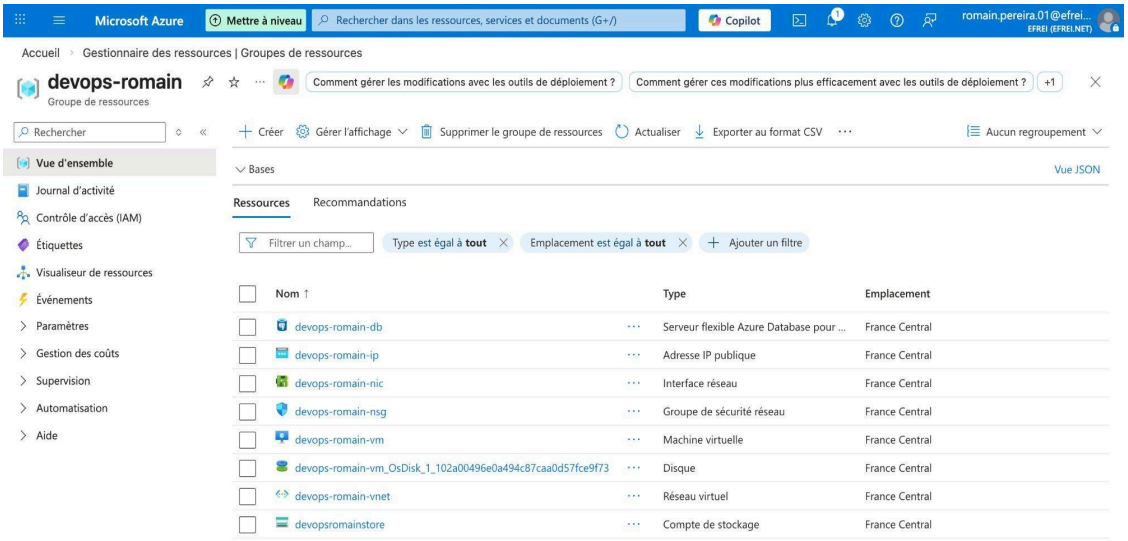
Solution : j'ai changé la région de West Europe vers France Central, mis la zone sur '2', et passé le SKU de l'IP publique en Standard. La taille de VM Standard_B1s n'étant pas non plus disponible dans cette région, j'ai utilisé Standard_D2s_v3 à la place.

Après ces corrections, le déploiement s'est terminé avec succès. Terraform affiche les outputs à la fin : l'IP publique de la VM, l'URL Flask, et le nom du storage account.

```
Apply complete! Resources: 1 added, 0 changed, 0 destroyed.

Outputs:

container_name = "fichiers"
flask_url = "http://4.233.124.251:5000"
storage_account_name = "devopsromainstore"
vm_ip_publique = "4.233.124.251"
```



4. Backend Flask

Le startup-script prévu dans la configuration Terraform pour installer Flask automatiquement n'a pas fonctionné. Une erreur dans cloud-init a empêché l'exécution du script au démarrage de la VM. J'ai donc installé les dépendances manuellement via SSH.

Les librairies installées sur la VM : flask, azure-storage-blob, psycopg2-binary.

L'application Flask est démarrée en tant que service systemd pour qu'elle redémarre automatiquement si la VM redémarre ou si le processus plante.

L'API expose 4 routes :

- GET / — vérifie que le serveur répond
- POST /upload — reçoit un fichier, l'envoie dans le Blob Storage, et enregistre les métadonnées dans PostgreSQL
- GET /fichiers — retourne la liste de tous les fichiers enregistrés en base
- DELETE /fichiers/<id> — supprime le fichier du Blob Storage et de la base de données

Problème 2 — Port 5000 inaccessible depuis l'extérieur

Flask tournait correctement sur la VM (vérifié en local sur 127.0.0.1), mais les requêtes depuis mon Mac échouaient. L'API ne répondait pas.

```
→  ~ curl http://4.233.124.251:5000/

curl: (52) Empty reply from server
→  ~
```

La cause : dans le Network Security Group défini dans main.tf, la règle firewall autorisait le port 8080 au lieu du port 5000. Flask écoute sur le port 5000 mais Azure bloquait le trafic entrant sur ce port. La correction dans main.tf puis un terraform apply ont mis à jour la règle.

```
→  ~ curl http://4.233.124.251:5000/
{"message":"Flask API fonctionne !","status":"ok"}
→  ~
```

```
adminuser@devops-romain-vm:~$ sudo systemctl status flask
● flask.service - Flask Application
     Loaded: loaded (/etc/systemd/system/flask.service; enabled; vendor preset: enabled)
     Active: active (running) since Fri 2026-03-13 11:06:40 UTC; 3s ago
   Main PID: 3575 (python3)
      Tasks: 1 (limit: 9524)
     Memory: 35.1M
        CPU: 448ms
     CGroup: /system.slice/flask.service
             └─3575 /usr/bin/python3 app.py
```

5. Tests CRUD

Les opérations CRUD ont été testées depuis mon terminal avec curl. Chaque opération passe par l'API Flask qui interagit avec Azure Blob Storage et PostgreSQL.

Upload d'un fichier — le fichier est envoyé au Blob Storage, un UUID est généré comme identifiant, et les métadonnées sont sauvegardées en base :

```
→  ~ curl -X POST http://4.233.124.251:5000/upload -F "file=@/tmp/test.txt"

{"id":"813e5ba9-4a18-4447-90f1-b0d1b4f21cd5","message":"Fichier uploade"}
→  ~
```

Liste des fichiers — retourne tous les fichiers enregistrés avec leur chemin, nom et date d'upload :

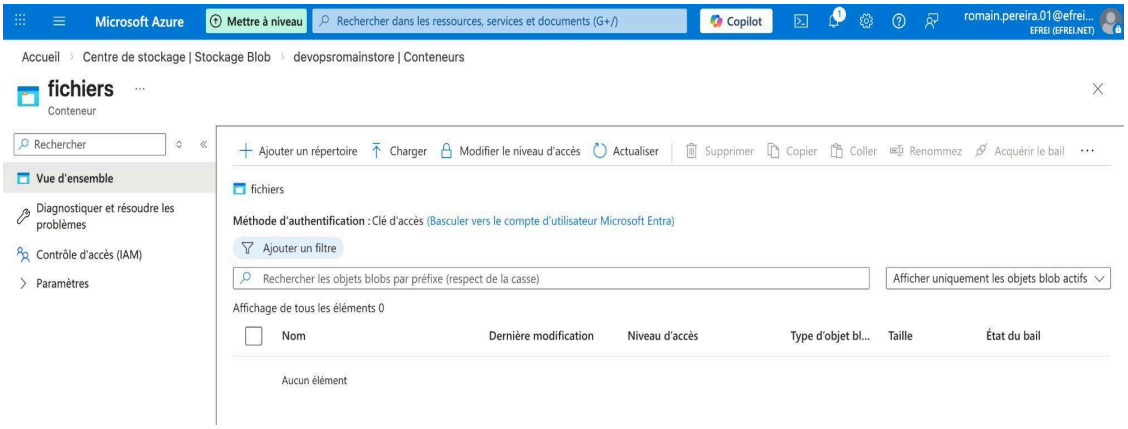
```
→  ~ curl http://4.233.124.251:5000/fichiers

[{"blob_path":"813e5ba9-4a18-4447-90f1-b0d1b4f21cd5_test.txt","filename":"test.t
xt","id":"813e5ba9-4a18-4447-90f1-b0d1b4f21cd5","uploaded_at":"2026-03-13T11:02:
32.018539"}]
→  ~
```

Suppression : le fichier est supprimé du Blob Storage et l'entrée est retirée de la base PostgreSQL :

```
→  ~ curl -X DELETE http://4.233.124.251:5000/fichiers/813e5ba9-4a18-4447-90f1-b
0d1b4f21cd5
{"message":"Fichier supprime"}
→  ~
```

Vérification dans le portail Azure, le container Blob est vide après la suppression, ce qui confirme que l'opération a bien fonctionné côté stockage :



6. Suppression de l'infrastructure

Une fois les tests terminés, la commande terraform destroy supprime toutes les ressources dans le bon ordre en gérant les dépendances. Les 12 ressources créées ont été supprimées en quelques minutes.

C'est un des grands avantages de Terraform : pas besoin de supprimer les ressources une par une sur le portail Azure. Une seule commande suffit.

```
azurerm_resource_group.main: Destruction complete after 1m16s

Destroy complete! Resources: 12 destroyed.
→  terraform
```

7. Conclusion

Ce projet m'a permis de mettre en pratique les concepts d'Infrastructure as Code avec Terraform sur Azure. J'ai déployé une vraie infrastructure avec VM, stockage cloud, base de données et backend API, le tout automatisé.

Les principales difficultés venaient des restrictions de l'abonnement Azure Efrei qui limitait certains SKUs et zones de disponibilité. Ces erreurs m'ont permis de mieux comprendre comment fonctionne la gestion des ressources Azure et comment les diagnostiquer.

Le gros avantage c'est que tout est reproductible : quelqu'un d'autre peut cloner le repo, mettre ses propres identifiants Azure dans terraform.tfvars, et avoir exactement la même infrastructure qui tourne en quelques minutes.